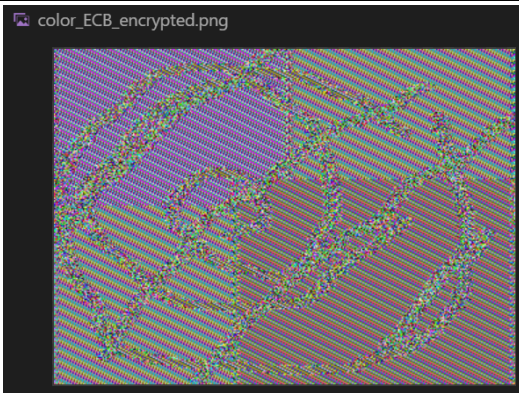
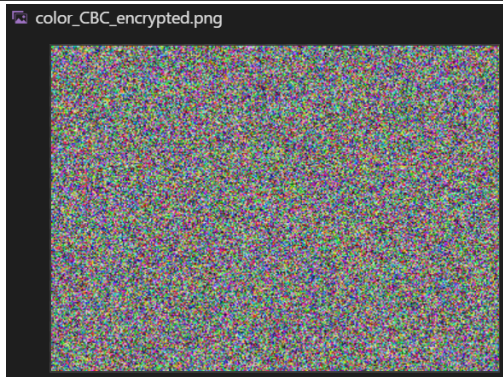
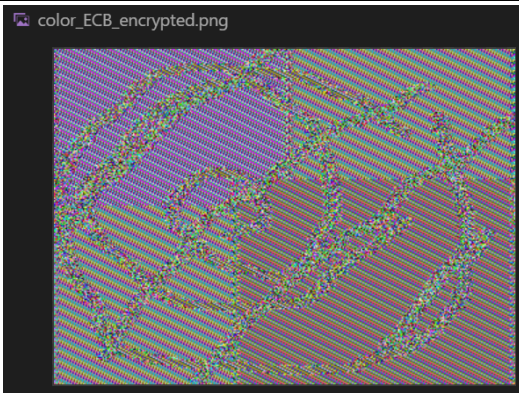
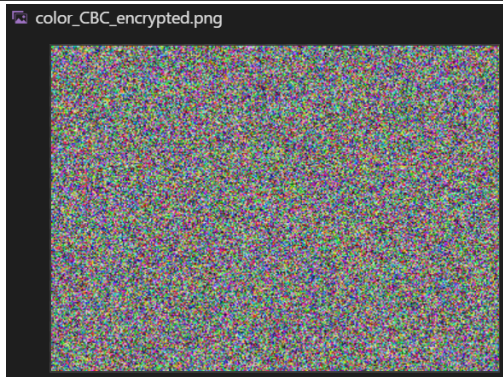
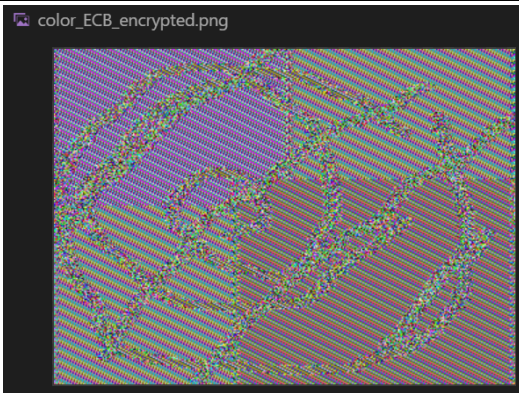
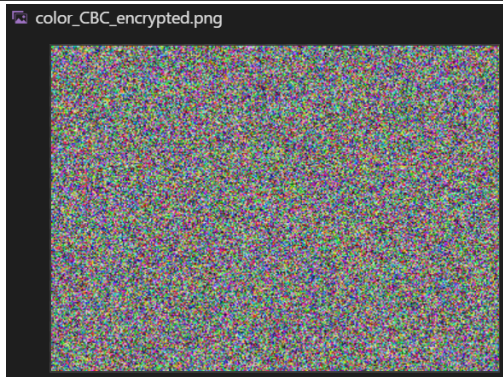
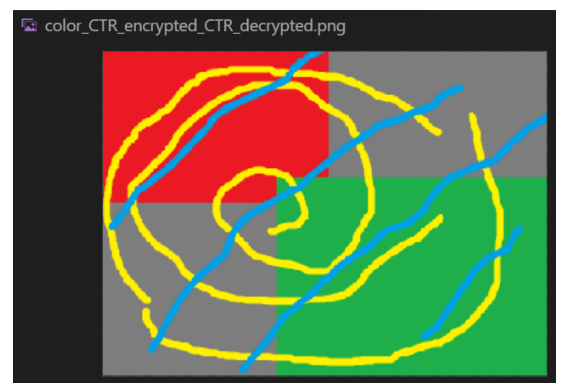


分工													
	Decrypt												
四資工三甲 B10632032 黃千綾	Main、Encrypt												
建置環境													
作業系統: Win 10 IDE: Visual Studio Code 程式語言: Python 3 套件: Pillow、pycryptodome													
操作方式													
可以用兩種方式執行: 1. 直接執行並輸入變數: - 圖檔路徑: PNG、BMP、JPEG及GIF均適用 - Block cipher mode: ECB/ CBC/ CTR - 金鑰: 16字元、不分大小寫 2. 使用參數, 例如: - python ImageEncrypt.py ./color.png CTR 0123456789ABCDEF - python ImageDecrypt.py ./color_ECB_encrypted.png ECB 0123456789ABCDEF 執行中會產生 _mode.ppm檔, 執行結束後 _mode.ppm檔會刪除, 並產生 _mode_encrypted 或 _mode_decrypted 的圖檔。													
執行結果截圖													
<table><tr><th>Key</th><th>Encrypt</th><th>Decrypt</th></tr><tr><td>0123456789ABCDEF</td><td></td><td></td></tr><tr><td>ECB</td><td></td><td></td></tr><tr><td>CBC</td><td></td><td></td></tr></table>	Key	Encrypt	Decrypt	0123456789ABCDEF			ECB			CBC			
Key	Encrypt	Decrypt											
0123456789ABCDEF													
ECB													
CBC													

CTR



程式碼解說

- main()

- 利用argv 或 input 輸入圖檔路徑、block cipher mode 和 key, 將key轉成大寫。

```

7 def main():
8     # 輸入圖檔路徑、block cipher mode 和 key
9     file_path = None
10    mode = None
11    key = None
12    if len(sys.argv) == 4:
13        file_path = sys.argv[1]
14        mode = sys.argv[2]
15        key = (sys.argv[3]).upper()
16    else:
17        file_path = input('Enter the path of the image (Ex. penguin.png): ')
18        mode = input(
19            'Enter the mode of block cipher (ECB/CBC/CTR): ')
20        key = (input('Enter the key (16 characters, not case-sensitive): ')).upper()

```

- 分別取得檔名及副檔名, 將圖檔轉存成 .ppm, 並對 .ppm 加密。

```

22    # 取出檔案名稱和副檔名
23    filename, ext = os.path.splitext(os.path.split(file_path)[1])
24
25    # 把檔案轉存成 ./____.ppm
26    Image.open(file_path).save(filename + '_' + mode + '.ppm')
27
28    # 對 .ppm 加密
29    if mode == 'ECB':
30        ECB(filename + '_' + mode + '.ppm', key)
31    elif mode == 'CBC':
32        CBC(filename + '_' + mode + '.ppm', key)
33    elif mode == 'CTR':
34        CTR(filename + '_' + mode + '.ppm', key)

```

- 將 .ppm 轉為圖檔 (依據之前取得的副檔名, 其中如果是 .jpg 必須改為 jpeg)

```

36    # 將 .ppm 轉存成 ./____.xxx (jpeg/gif/png/bmp)
37    target_format = ext[1:]
38    if target_format.lower() == 'jpg':
39        target_format = 'jpeg'
40    Image.open(filename + '_' + mode + '.ppm').save(filename + '_' + mode +
41        '_encrypted' + ext, target_format)
42
43    # 刪除 ./____.ppm
44    os.remove(filename + '_' + mode + '.ppm')
45    # // end main()

```

- ECB(file_path, key)

- Encrpyt

- ◆ 讀取檔案並照抄前三行(.ppm的檔案格式描述)。

```
48 def ECB(file_path, key):
49     ... result = b''
50     ... with open(file_path, 'rb') as file:
51         ...     # 照抄前三行
52         ...     enter_amount = 0
53         ...     while(enter_amount != 3):
54             ...         temp = file.read(1)
55             ...         result += temp
56         ...         if(temp == b'\n'):
57             ...             enter_amount += 1
```

- ◆ 以16 bytes 為 block, 用AES加密。

```
58     ...     # 每個 block 用 AES 加密
59     ...     cipher = AES.new(bytes(key.encode('utf-8')), AES.MODE_ECB)
60     ...     current_block = file.read(16)
61     ...     while(len(current_block) == 16):
62         ...         ciphertext = cipher.encrypt(current_block)
63         ...         result += ciphertext
64         ...         current_block = file.read(16)
```

- ◆ 若current_block 有未滿16 bytes 的內容, 則用PKCS的方式補滿, 並加密;最後將 result 寫進 .ppm 中。

```
66     ...     if(len(current_block) != 0):
67         ...         padding_amount = 16 - len(current_block)
68         ...         padding = (padding_amount*16).to_bytes(1, 'big')
69         ...         current_block += padding * padding_amount
70         ...         ciphertext = cipher.encrypt(current_block)
71         ...         result += ciphertext
72     ...     with open(file_path, 'wb') as file:
73         ...         file.write(result)
74     # // end ECB()
```

■ Decrypt

- ◆ 讀取檔案並照抄前三行。

```
48 def ECB(file_path, key):
49     ... result = b''
50     ... with open(file_path, 'rb') as file:
51         ...     line = 1
52         ...     while line <= 3:
53             ...         one_byte = file.read(1)
54             ...         result += one_byte
55             ...         if one_byte == b'\n':
56                 ...             line += 1
```

- ◆ 以16 bytes 為 block, 用AES解密。

```
57     ...     cryptor = AES.new(bytes(key.encode('utf-8')), AES.MODE_ECB)
58     ...     current_block = file.read(16)
59     ...     while len(current_block) == 16: ...     # 16bytes
60         ...         plain_text = cryptor.decrypt(current_block)
61         ...         result += plain_text
62         ...         current_block = file.read(16)
```

- ◆ 若current_block 有未滿16 bytes 的內容, 則用PKCS的方式補滿, 並解密;最後將 result 寫進 .ppm 中。

```

63     ....if len(current_block) != 0: .....# 不夠16bytes
64     ....padding_amount = padding_amount: int (len_block)
65     ....padding = padding_amount.to_bytes(1, 'little')
66     ....current_block += padding * padding_amount
67     ....plain_text = cryptor.decrypt(current_block)
68     ....result += plain_text
69     ....with open(file_path, 'wb') as file:
70     ....file.write(result)

```

- CBC(file_path, key)

- Encrypt

- ◆ 產生IV。

```

78     ....# 產生IV
79     ....i_v = ''
80     ....i_p = [7, 15, 6, 1, 10, 8, 3, 11, 16, 13, 2, 9, 14, 4, 5, 12]
81     ....i_p = i_p[(int(key[0]) % 16):] + i_p[:int(key[0]) % 16]
82     ....for index in i_p:
83     ....i_v += key[index - 1]
84     ....i_v = bytes(i_v.encode('utf-8'))

```

- ◆ 讀取檔案並照抄前三行(程式碼同ECB)。

- ◆ 以16 bytes 為 block, 先與IV做XOR, 再用AES加密;並將結果做為下一個迴圈的IV。

```

94     ....# 每個block用AES加密
95     ....cipher = AES.new(bytes(key.encode('utf-8')), AES.MODE_ECB)
96     ....current_block = file.read(16)
97     ....while(len(current_block) == 16):
98     ....current_block = bytes([current_block[i] ^ i_v[i]
99     ....for i in range(16)])
100    ....ciphertext = cipher.encrypt(current_block)
101    ....# 以密文作為下一個IV
102    ....i_v = ciphertext
103    ....result += ciphertext
104    ....current_block = file.read(16)

```

- ◆ 若current_block 有未滿16 bytes 的內容, 則用PKCS的方式補滿, 並同上步驟;最後將result 寫進 .ppm 中。

```

105    ....# PKCS padding
106    ....if(len(current_block) != 0):
107    ....padding_amount = 16 - len(current_block)
108    ....padding = (padding_amount*16).to_bytes(1, 'big')
109    ....current_block += padding * padding_amount
110    ....current_block = bytes([current_block[i] ^ i_v[i]
111    ....for i in range(16)])
112    ....ciphertext = cipher.encrypt(current_block)
113    ....result += ciphertext
114    ....with open(file_path, 'wb') as file:
115    ....file.write(result)
116    # // end CBC()

```

- Decrypt

- ◆ 產生IV(程式碼同Encrypt)。

- ◆ 讀取檔案並照抄前三行(程式碼同ECB)。

- ◆ 以16 bytes 為 block, 先與IV做XOR, 再用AES解密;並將結果做為下一個迴圈的IV。

```

92 ..... cipher_block = file.read(16)
93 ..... # 16 bytes
94 ..... while len(cipher_block) == 16:
95 .....     plain_text = cryptor.decrypt(cipher_block)
96 .....     plain_text = bytes([plain_text[i] ^ i_v[i] for i in range(16)])
97 .....
98 .....     # 以密文作為下一個 IV
99 .....     i_v = cipher_block
100 .....     result += plain_text
101 .....     cipher_block = file.read(16)

```

- ◆ 若cipher_block 有未滿16 bytes 的內容, 則用PKCS的方式補滿, 並同上步驟;最後將result 寫進 .ppm 中。

```

102 ..... # 不夠 16 bytes, PKCS padding
103 ✓ ..... if len(cipher_block) != 0:
104 .....     padding_amount = 16 - len(cipher_block)
105 .....     padding = padding_amount.to_bytes(1, 'little')
106 .....     cipher_block += padding * padding_amount
107 .....     plain_text = cryptor.decrypt(cipher_block)
108 .....     plain_text = bytes([plain_text[i] ^ i_v[i] for i in range(16)])
109 .....     result += plain_text
110 ✓ ..... with open(file_path, 'wb') as file:
111 .....     file.write(result)

```

- CTR(file_path, key)

- Encrypt

- ◆ 產生IV(程式碼同CBC)。
 - ◆ 讀取檔案並照抄前三行(程式碼同ECB)。
 - ◆ 將IV以AES加密後, 與16 bytes的明文做XOR後放入result;並將IV加一。

```

137 ..... # 每個 block 用 AES 加密
138 ..... cipher = AES.new(bytes(key.encode('utf-8')), AES.MODE_ECB)
139 ..... current_block = file.read(16)
140 ..... while len(current_block) == 16:
141 .....     ciphertext_i_v = cipher.encrypt(i_v)
142 .....     ciphertext = bytes(
143 .....         [current_block[i] ^ ciphertext_i_v[i] for i in range(16)])
144 .....     result += ciphertext
145 .....     current_block = file.read(16)
146 .....     # IV += 1
147 .....     i_v = (int.from_bytes(i_v, 'big') + 1).to_bytes(16, 'big')

```

- ◆ 若current_block 有未滿16 bytes 的內容, 則用PKCS的方式補滿, 並同上步驟;最後將result 寫進 .ppm 中。

```

148 ..... # PKCS padding
149 ..... if len(current_block) != 0:
150 .....     padding_amount = 16 - len(current_block)
151 .....     padding = (padding_amount*16).to_bytes(1, 'big')
152 .....     current_block += padding * padding_amount
153 .....     ciphertext_i_v = cipher.encrypt(i_v)
154 .....     ciphertext = bytes([current_block[i] ^ ciphertext_i_v[i]
155 .....         for i in range(16)])
156 .....     result += ciphertext
157 ..... with open(file_path, 'wb') as file:
158 .....     file.write(result)
159 # // end CTR()

```


■ Decrypt

- ◆ 產生IV(程式碼同Encrypt)。
- ◆ 讀取檔案並照抄前三行(程式碼同ECB)。
- ◆ 將IV以AES加密後, 與16 bytes的明文做XOR後放入result; 並將IV加一。

```

132     cryptor = AES.new(bytes(key.encode('utf-8')), AES.MODE_ECB)
133
134     cipher_block = file.read(16)
135     while len(cipher_block) == 16:
136         cipher_i_v = cryptor.encrypt(i_v)
137         plain_text = bytes([cipher_block[i] ^ cipher_i_v[i]
138                             for i in range(16)])
139         result += plain_text
140         i_v = (int.from_bytes(i_v, 'big') + 1).to_bytes(16, 'big')
141         cipher_block = file.read(16)

```

- ◆ 若current_block 有未滿16 bytes 的內容, 則用PKCS的方式補滿, 並同上步驟;最後將result 寫進 .ppm 中。

```

142     # 不夠 16 bytes, PKCS padding
143     if len(cipher_block) != 0:
144         padding_amount = 16 - len(cipher_block)
145         padding = padding_amount.to_bytes(1, 'little')
146         cipher_block += padding * padding_amount
147         cipher_i_v = cryptor.encrypt(i_v)
148         plain_text = bytes([cipher_block[i] ^ cipher_i_v[i]
149                             for i in range(16)])
150         result += plain_text
151     with open(file_path, 'wb') as file:
152         file.write(result)

```

遇到困難與心得

在執行速度的部分, 使用網路上的企鵝圖來測試時, 因為檔案比較大, 執行時間非常久。一度想要使用Thread 和 Lock 來平行處理, 讓每個block可以在不同的執行緒上處理, 但是似乎在操作檔案的時候還是有所衝突, 所以決定放棄這個部分。

關於程式語言, 之前的作業都是用C++寫, 後來發現用Python真的比較方便, 所以改用Python。但是一開始不太熟悉Python的語法, 所以會在一些基本的操作和轉換型別發生很多錯誤;在命名規範上也和組員不一致。後來查文件之後, 就有比較順利。此外在安裝套件時也有出現一些小錯誤(例如套件資料夾名稱的問題), 但在爬文後就解決了。

在轉檔的部分, 發現Python可以import os, 很簡單地處理檔案路徑, 程式碼也很簡潔。

關於Crypto套件, 在設定AES的MODE時, 一開始很直覺地就根據實作的mode來選擇, 卻發現結果是錯的。思考之後了解到, 雖然是實作CBC或CTR, 但因為只是要單純用AES幫block加密, 所以要選MODE_ECB, 不然block會被用CBC或CTR的方式處理。